



DEVOXX 2026

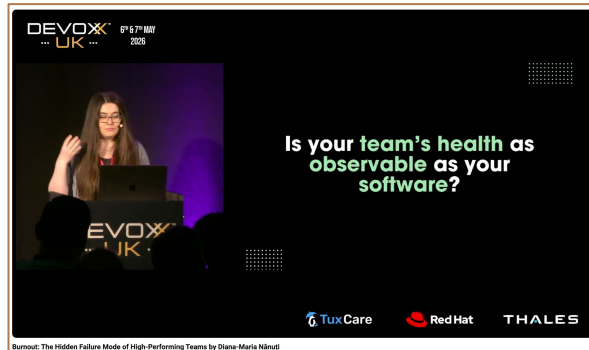
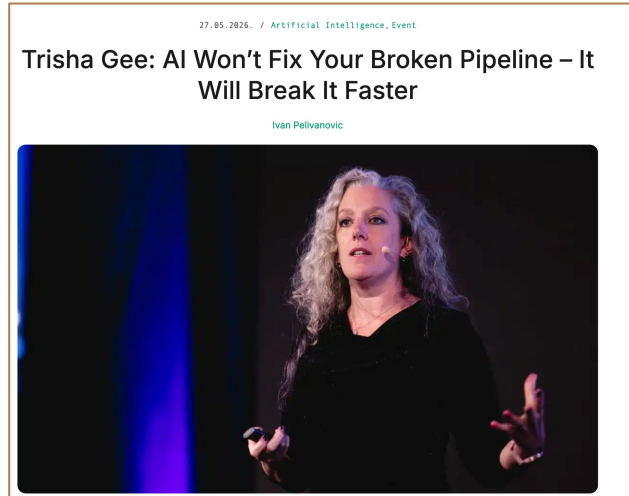
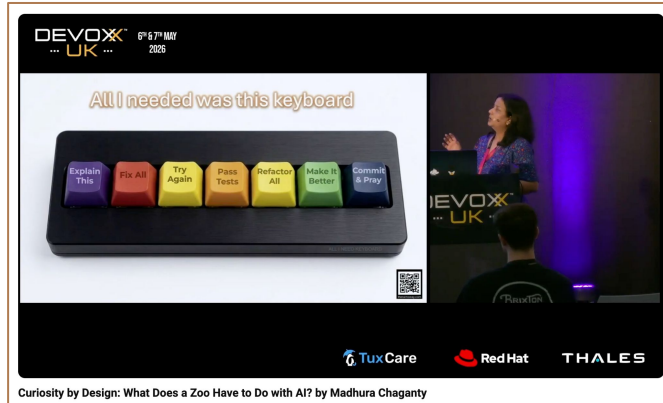
Feature	Devoxx UK	QCon (London)	Spring I/O
Primary Audience	Java Developers & Tech Enthusiasts	Senior Engineers & Architects	Spring/Java Ecosystem
Typical Attendees	~1,000	~1,000	~1,300+
Focus	Community, Education, Depth, Breadth	Architecture, Leadership, Depth	Framework Mastery, Ecosystem
Key Topics	Java, AI/ML, Cloud, DevOps, Architecture, Web	Distributed Systems, AI Engineering, Culture, Cloud, Security	Spring Boot, Spring Cloud, AI Integration, Java, Persistence
Vibe	Community-led, High Energy	Professional, Peer-Led, Pragmatic	Technical, Specialized, Expert

Main themes:

- **Teams & Individual productivity:** nice to catch up with the industry
- **Hardcore: newest Java features & Multithreading:** no AI, only hardcore
- **AI tried by Java Champions:** how to run models locally, how to build pipeline and (already?!) antipatterns
- **Architecture & Testing**
- **Intellij IDEA:** productivity tips
- **Open Source:** Spring AI, Quarkus, Langchain4j

Devoxx UK 2026 featured over 100 sessions and admitted 300+ attendees

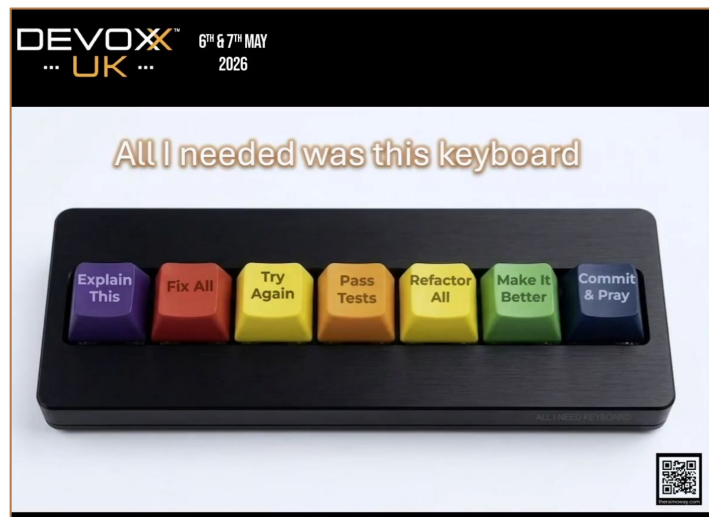
Team & Individual Productivity



Curiosity by Design

To keep engineering instincts sharp, she proposes the WILD framework:

1. **W - Welcome Friction:** Intentionally introduce moments of difficulty—like reading documentation or manually troubleshooting—to ensure you own the solution rather than just hitting "fix all."
2. **I - Investigate the Why:** Move beyond asking "Does it work?" to asking "Why does it work?" and looking for better design patterns during code reviews.
3. **L - Limit the Feed:** Dedicate time for "no-AI" sessions, such as open-mic discussions or whiteboarding, to encourage independent thinking and design.
4. **D - Debrief the Hunt:** Treat AI interactions like post-mortems for production incidents. Analyze what went well and what went wrong during the coding process.



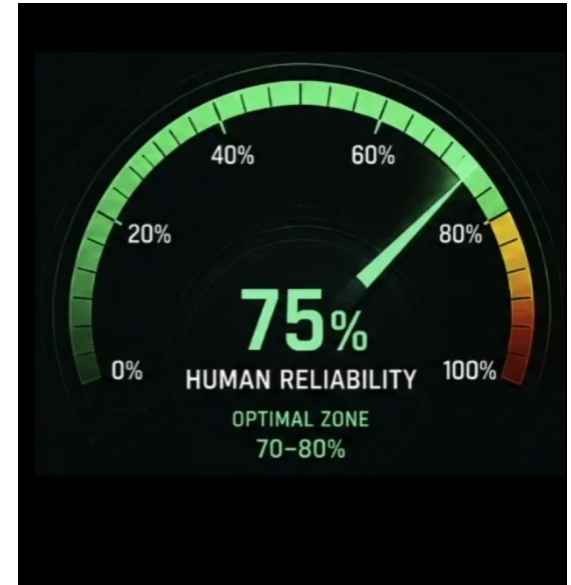
Shiny New Tools Won't Fix Your Problem

- **AI is an amplifier:** It enhances existing strengths but accelerates existing dysfunctions.
- **Coding isn't the bottleneck:** Maintenance, testing, and deployment remain the hard, expensive parts.
- **Fundamentals still matter:** Quality design and clean code are essential safety nets.
- **Focus on real value:** Prioritize process improvement over simple generation speed.



Burnout: The Hidden Failure Mode of High-Performing Teams

1. **Burnout as a Systemic Failure:** When we run teams at 100% capacity without "safety margins," we create a high-throughput system destined for failure, much like poorly engineered software
2. **The Need for Human Observability:** Just as we track system metrics like CPU or latency, we must instrument our teams to monitor indicators like cognitive load, context switching, and emotional stress to detect burnout before it causes an outage
3. **Defining Leading Indicators:** Teams should track metrics such as "Human MTTR" (using quick pulse surveys), "Verification Load" (debugging AI work), and "Human Error Budgets" to determine if the team is operating sustainably
4. **Prioritizing Resilience Over Heroics:** To ensure longevity, teams must build redundancy to eliminate single points of failure, maintain "slack" or idle capacity (aiming for 70-80% utilization), and enforce hard boundaries or "circuit breakers" for recovery



Hardcore

DEVOXX ... UK ... 6th & 7th MAY 2026

How to determine the optimal number of threads to pick when doing some repeated number of async tasks?

How Many Threads Is Too Many?

Optimal number of threads per core

What's your favourite number of threads? by Ryan Healey

TuxCare Red Hat THALES

DEVOXX ... UK ... 6th & 7th MAY 2026

Rust's Thing: Zero-Cost Abstractions

like ...

- Memory safety is statically checked
- C: unsafe, segfaults at runtime
- Java: safe, but pays GC cost at runtime
- Rust: safe, proved at compile time
- Generics are *monomorphized* (like C++, unlike Java)
- Traits are statically dispatched by default
- Iterators compile to the same code as a hand-written loop
- Option<T> / Result<T,E> — no heap allocation

10 Things Rust Could Learn from Java (and 5 Things Java Should Probably Steal Back) by Scott Gerring

TuxCare Red Hat THALES

DEVOXX

Structured Concurrency: Examples

All these examples can be implemented using StructuredTaskScope API.

Google Cloud ING ORACLE AWS JETBRAINS

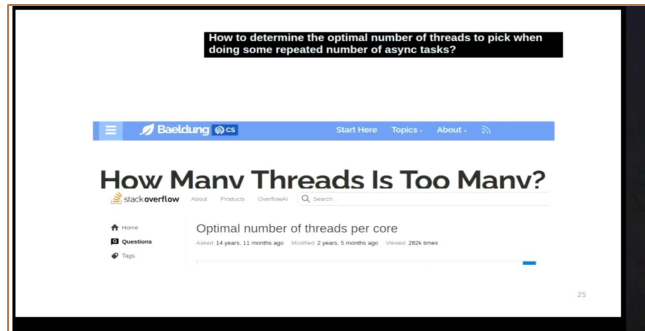
Virtual Threads, Structured Concurrency and Scoped Values: Putting it all together by Balakrishna R

Virtual Threads

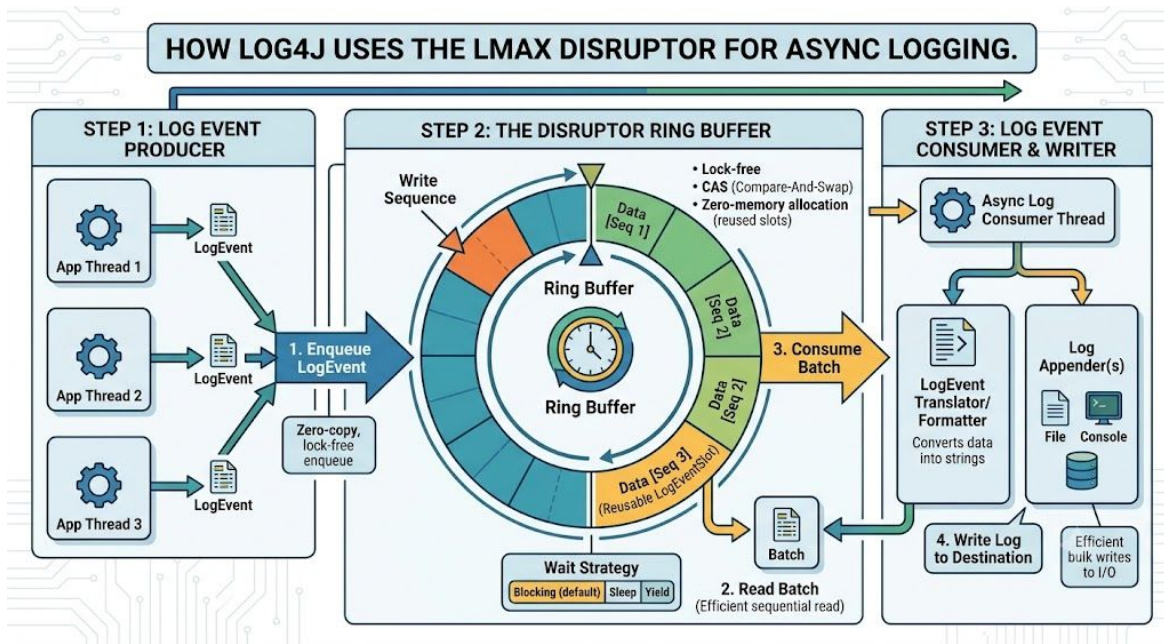
- **Virtual Threads:** Unlike traditional platform threads that have a 1:1 mapping with OS threads and require expensive pooling, virtual threads are lightweight user-mode threads. They enable high scalability by unmounting from carrier threads during blocking I/O operations, allowing for massive concurrency with minimal memory overhead.
- **Structured Concurrency:** This approach treats multiple concurrent tasks as a single unit of work, improving code readability and error handling. The speaker demonstrates this using `StructuredTaskScope`, highlighting benefits like automatic cancellation propagation, short-circuiting when a task fails, and a cleaner, imperative coding style compared to `CompletableFuture`
- **Scoped Values:** A modern alternative to `ThreadLocal`, scoped values provide a safer, immutable way to share data across threads. They solve issues like unconstrained mutability and unbounded lifetimes, and they integrate seamlessly with structured concurrency by being automatically inherited by child virtual threads.

What is your favorite number of threads?

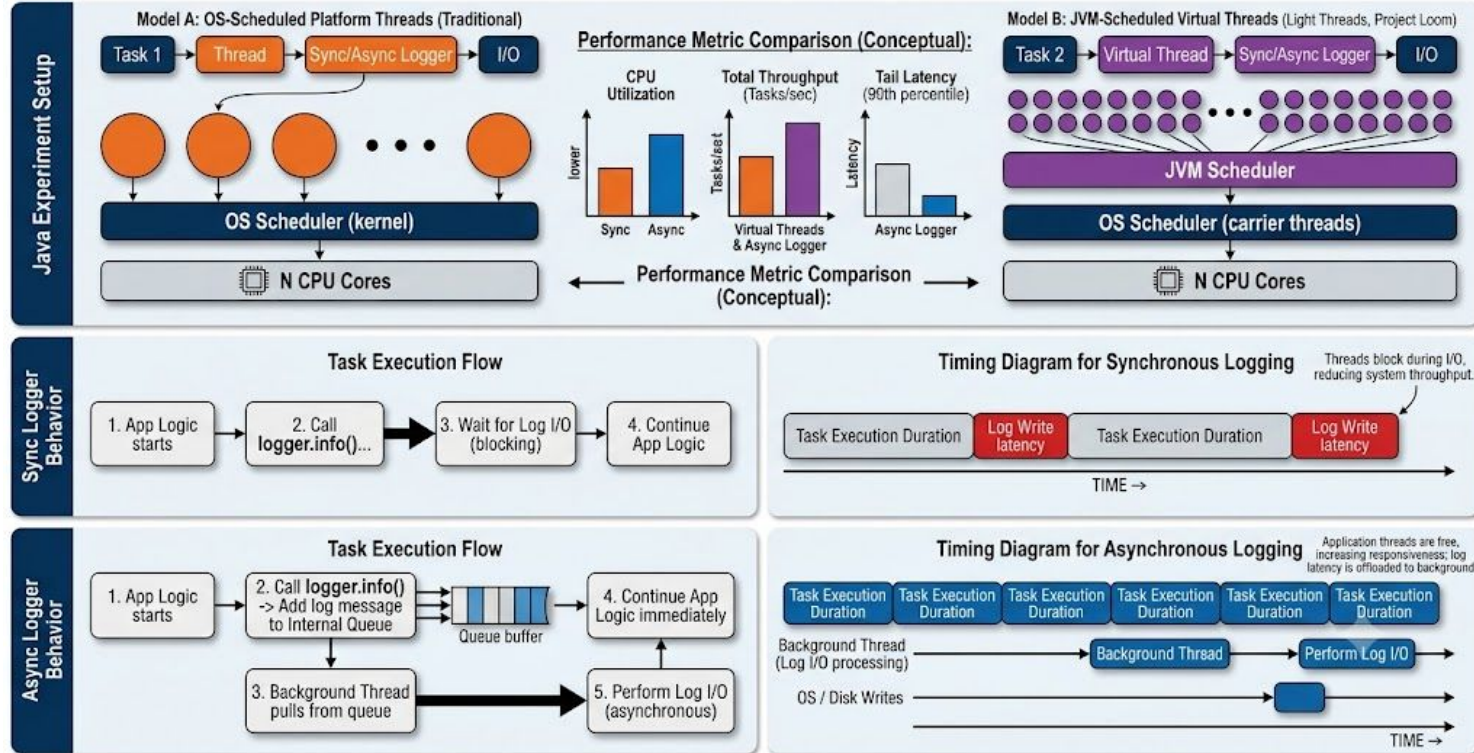
- **Defining the "Optimal" Number:** simply increasing thread counts frequently creates more overhead rather than speed.
- **The Reality of Virtual Threads:** Addresses the introduction of *Virtual Threads* in Java, explaining that while they are much more lightweight and efficient for high-concurrency I/O tasks, they **do not magically solve resource constraints** or eliminate the need for careful system design
- **The Case for Single-Threading:** Drawing from his experience at *Elmax Group*, he discusses how ultra-low latency systems—such as **the Disruptor pattern**—often rely on single-threaded execution to maximize cache efficiency and eliminate the complexity of thread orchestration
- **The "Human" Cost of Multi-threading:** A central argument of the talk is that multi-threaded code is inherently difficult to write, test, debug, and maintain. He reminds the audience that developers themselves are not multi-core processors, and excessive concurrency increases the cognitive load and attack surface of a codebase.



Hint: you're likely using Disruptor everyday



Multithreading & Light Threads Experimentation with Synchronous & Asynchronous Java Loggers



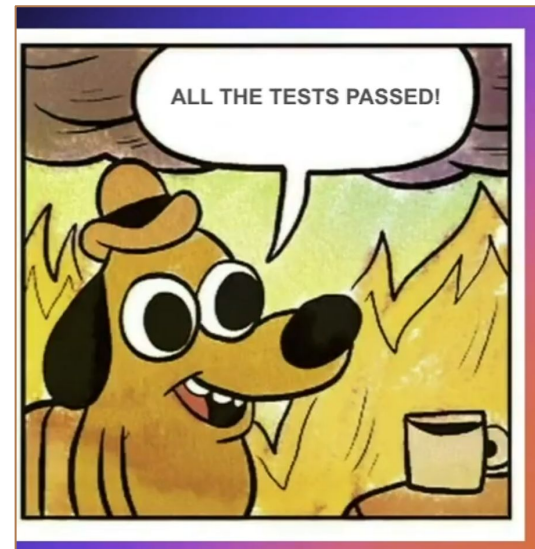
Speed, Stress, and Soak: Performance Testing

Main Types of Performance Testing:

- **Load Testing:** Checks how the system behaves under expected, predictable traffic (baseline).
- **Stress Testing:** Pushes the system beyond its limits (150-300% load) to find breaking points and assess recovery capabilities.
- **Spike Testing:** Tests the system's reaction to sudden, massive surges in traffic (e.g., 300% in seconds).
- **Soak/Endurance Testing:** Runs the system at average load for extended periods (days) to identify issues like memory leaks or resource exhaustion.
- **Capacity Testing:** Uses incremental load steps to plan for future growth and identify infrastructure ceilings.

Additional Testing Approaches:

- **Smoke Testing:** A simple sanity check for basic functionality, often run in CI pipelines.
- **Breakpoint Testing:** Continuously increases load until the system fails to identify the absolute hard limit.



AI Hype

DEVOXX UK 6th 7th MAY 2026

```
service Claude-skip-permissions Claude Code v2.1.119  
open v2.1.119 (selected) with high effort - Claude Team  
~/Devoxx/UK/2026/The Right 300 Tokens Beat 100k Noisy Ones /demo/service
```

The confire_order endpoint in app/ord

```
** bypass permissions on (shift+tab to cycle) @high / /effort
```

TuxCare Red Hat THALES

The Right 300 Tokens Beat 100k Noisy Ones: Four Context Antipatterns That Kill Your AI Agent

DEVOXX UK 6th 7th MAY 2026

Reliability Benchmarking

- > Standard LLM: 75% Consistency
- > Agentic framework: 88% Consistency
- > Sandwich Pattern: 99.9999% Reliability

Combining AI reasoning with deterministic flow is the only way to reach "Six Nines" reliability.

THALES

TuxCare Red Hat THALES

Developing AI-Centric Applications for Mission-Critical Environments by Ian Callaghan

DEVOXX UK 6th 7th MAY 2026

Local Development in the AI Era by Cedric Clyburn & Kevin Dubois

datasets, users...

Models 2,855,432 Filter by name

deepseek-ai/DeepSeek-V4-Pro
Text Generation · 862B · Updated 1 day ago · 946k · 3.7k

openai/privacy-filter
Token Classification · 1B · Updated 15 days ago · 165k · 1.34k

TuxCare Red Hat THALES

AI Hype

Core Engineering Principles:

- Deterministic Cages: Because AI models are inherently *stochastic* (probabilistic), they must be wrapped in a deterministic architecture to ensure predictable, safe behavior (0:42-10:40).
- The Sandwich Architecture: A three-layer system is used to gatekeep AI decisions (12:31-14:00):
 1. Pre-processing: Clean inputs and define hard-coded constraints.
 2. AI Engine: The non-deterministic model performs reasoning.
 3. Post-processing: Validates output against strict safety rules before action.
- Human Oversight Models: The decision to keep a human *in* the loop (active approval) or *on* the loop (veto-based oversight) depends on the cost of failure and the required reaction speed (20:56-22:54).

Safety & Verification:

- Red Teaming: Instead of standard unit tests, systems face thousands of adversarial scenarios designed to force safety breaches (23:38-25:50).
- Shadow Mode: New models are run in real-world environments without controlling hardware to compare their decisions against human or legacy benchmarks (32:42-33:52).
- Stochastic Diagnosis: Engineers maintain rigorous logs of prompts, seeds, and reasoning chains to ensure forensic auditability when probabilistic errors occur (33:52-35:28).

Callaghan concludes that the future of mission-critical AI is not just about intelligence, but about the deliberate engineering of trust (38:46-40:33).

Local Development in the AI Era

- **Run Locally:** You can run AI models on your own hardware using tools like **Ollama**, **LM Studio**, or **RamaLlama** (which runs in containers for easier deployment)
- **Right Tool for the Job:** Select models based on use case; use generalized models for reasoning and specialized models like *Qwen* or *Granite* for coding tasks. Tools like LLM Fit help determine if a model fits your hardware
- **Workflow Integration:** Use IDE/CLI tools like OpenCode or Continue. To optimize results, provide a `claude docs` or `agents.md` file in your repository to give the AI crucial project context
- **Hybrid Strategy:** Use a powerful frontier model for high-level architectural planning and delegate smaller, specific tasks to efficient local models
- **Development Reality:** **Smaller** local models may **hallucinate more** or struggle with complex, multi-step agentic tasks. They work best for specific, focused coding problems

Local Development in the AI Era

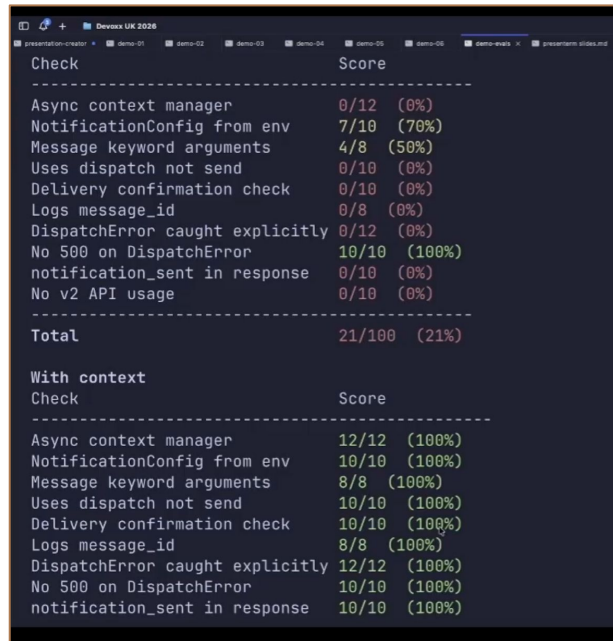
WHY???

- **Cost Management:** Frontier models are becoming increasingly expensive. The speakers point out that high-usage development workflows—such as using code assistants throughout the day—can lead to significant expenses (reaching upwards of \$100 for heavy users) compared to the zero-cost inference of local, open-weights models
- **Independence and Reliability:** You avoid vendor lock-in. If a cloud service experiences outages or changes its pricing model (such as the recent discussions around removing pro plans), your development workflow remains unaffected if it is based on local models
- **Privacy and Security:** By running models locally, you keep your code and data on your own hardware. This prevents sensitive proprietary or personal information from being sent to third-party servers to be potentially used for training future models
- **Latency and Offline Access:** Local models provide instant responses and work in network-optional environments. This is particularly valuable for developers who travel or work in areas with unreliable internet connections
- **Environmental Impact:** The presenters acknowledge the significant environmental cost of large data centers, including high energy consumption and water usage. Running smaller, specialized models locally is presented as a way to potentially mitigate your individual carbon footprint

Right 300 Tokens Beat 100k Noisy Ones

The Four Context Antipatterns

1. **The Stuffed Prompt:** Cramming excessive, conflicting, or unorganized information into a system prompt leads to failure. (*Irina: MCP actually there right now!*)
 - The Fix: **Use Skills**—modular, progressively disclosed context artifacts that are loaded only when needed.
2. **The Wrong Tool for the Job :** Relying solely on RAG (**Retrieval-Augmented Generation**) based on similarity often leads to incorrect data selection.
3. **The Goldfish Agent:** LLMs are stateless, and relying on black-box memory or compaction causes agents to "forget" important decisions.
 - The Fix: Implement external, versioned memory stored directly in your codebase, giving you full control over what is saved and retrieved.
4. **Vibes Eval:** poorly designed automated tests creates a false sense of security.
 - The Fix: Implement rigorous Evaluations. Have the LLM generate scenarios and rubrics, review them with human expertise, run them to assess the delta, and continuously monitor for regressions when updates occur.

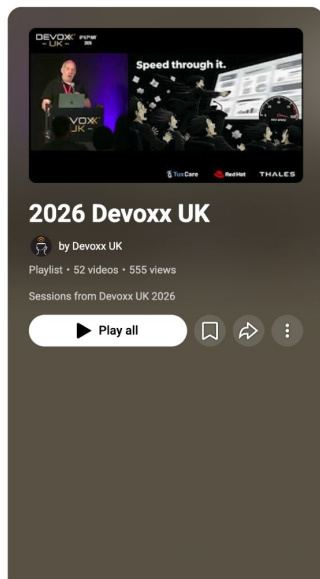


The screenshot shows a terminal window with a dark background. At the top, there's a title bar with icons and the text 'Demo-05'. Below it, a list of tabs includes 'presentation-creator', 'demo-01', 'demo-02', 'demo-03', 'demo-04', 'demo-05' (selected), 'demo-06', 'demo-07', and 'presentation-slides'. The main content area displays test results for 'demo-05'.

Check	Score
Async context manager	0/12 (0%)
NotificationConfig from env	7/10 (70%)
Message keyword arguments	4/8 (50%)
Uses dispatch not send	0/10 (0%)
Delivery confirmation check	0/10 (0%)
Logs message_id	0/8 (0%)
DispatchError caught explicitly	0/12 (0%)
No 500 on DispatchError	10/10 (100%)
notification_sent in response	0/10 (0%)
No v2 API usage	0/10 (0%)
Total	21/100 (21%)

With context	Score
Async context manager	12/12 (100%)
NotificationConfig from env	10/10 (100%)
Message keyword arguments	8/8 (100%)
Uses dispatch not send	10/10 (100%)
Delivery confirmation check	10/10 (100%)
Logs message_id	8/8 (100%)
DispatchError caught explicitly	12/12 (100%)
No 500 on DispatchError	10/10 (100%)
notification_sent in response	10/10 (100%)

Where to find talks:



2026 Devovx UK

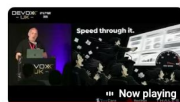
by Devovx UK

Playlist • 52 videos • 555 views

Sessions from Devovx UK 2026

▶ Play all

🔖 🔄 ⋮



How to Crash & Burn in 7 Minutes by Steve Poole. An Aspiring Speakers Bonus Session

Devovx UK
218 views • 1 month ago



Balancing immediate value delivery with sustainable long term capabilities by Franco Cassar Manghi

Devovx UK
65 views • 1 month ago



Aspiring Speakers: From Replacement to Rocket Fuel: Launching Your Tech Career by Sudi Mandyam

Devovx UK
154 views • 1 month ago



Aspiring Speakers: Learning Python to buy shoes by Isaac Oldwood

Devovx UK
62 views • 1 month ago



Aspiring Speakers Session: Cloud, Code, and Curiosity by Omodolapo Babatunde

Devovx UK
47 views • 1 month ago

